

A generative adversarial network (GAN) is a class of machine learning frameworks and a prominent framework for approaching generative AI. The concept was initially developed by Ian Goodfellow and his colleagues in June 2014. In a GAN, two neural networks compete with each other in the form of a zero-sum game, where one agent's gain is another agent's loss.

Given a training set, this technique learns to generate new data with the same statistics as the training set. For example, a GAN trained on photographs can generate new photographs that look at least superficially authentic to human observers, having many realistic characteristics. Though originally proposed as a form of generative model for unsupervised learning, GANs have also proved useful for semi-supervised learning, fully supervised learning, and reinforcement learning.

The core idea of a GAN is based on the "indirect" training through the discriminator, another neural network that can tell how "realistic" the input seems, which itself is also being updated dynamically. This means that the generator is not trained to minimize the distance to a specific image, but rather to fool the discriminator. This enables the model to learn in an unsupervised manner.

GANs are similar to mimicry in evolutionary biology, with an evolutionary arms race between both networks.

Definition

Mathematical

The original GAN is defined as the following game:

Each probability space

(

Ω

,

μ

ref

)

$$\{\Omega, \mu_{\text{ref}}\}$$

defines a GAN game.

There are 2 players: generator and discriminator.

The generator's strategy set is

$\mathcal{P}$

(

Ω

)

$$\{\mathrm{mathcal{P}}(\Omega)\}$$

, the set of all probability measures

μ

G

$$\mu_G$$

on

Ω

$$\{\displaystyle \Omega \}$$

.

The discriminator's strategy set is the set of Markov kernels

$$\mu$$

$$D$$

$$:$$

$$\Omega$$

$$\rightarrow$$

$$P$$

$$[$$

$$0$$

$$,$$

$$1$$

$$]$$

$$\mu_D:\Omega \rightarrow \mathcal{P}[0,1]$$

, where

$$P$$

$$[$$

$$0$$

$$,$$

$$1$$

$$]$$

$$\mathcal{P}[0,1]$$

is the set of probability measures on

$$[0,1]$$

$$\{ \displaystyle [0,1] \}$$

.

The GAN game is a zero-sum game, with objective function

$$L(\mu, G)$$

$$\mu$$

$$G$$

$$,$$

$$\mu$$

D

)

:=

E

x

~

μ

ref

,

y

~

μ

D

(

x

)

[

ln

y

]

+

E

x

~

μ

G

,

y

~

μ

D

(

x

)

[

ln

(

1

-

y

)

]

.

$$L(\mu_{\text{G}}, \mu_{\text{D}}) := \operatorname{E}_{x \sim \mu_{\text{ref}}, y \sim \mu_{\text{D}}(x)} [\ln y] + \operatorname{E}_{x \sim \mu_{\text{G}}, y \sim \mu_{\text{D}}(x)} [\ln(1-y)].$$

The generator aims to minimize the objective, and the discriminator aims to maximize the objective.

The generator's task is to approach

μ

G

$\approx$

μ

ref

$$\mu_G \approx \mu_{\text{ref}}$$

, that is, to match its own output distribution as closely as possible to the reference

distribution. The discriminator's task is to output a value close to 1 when the input appears to be from the reference distribution, and to output a value close to 0 when the input looks like it came from the generator distribution.

In practice

The generative network generates candidates while the discriminative network evaluates them. This creates a contest based on data distributions, where the generator learns to map from a latent space to the true data distribution, aiming to produce candidates that the discriminator cannot distinguish from real data. The discriminator's goal is to correctly identify these candidates, but as the generator improves, its task becomes more challenging, increasing the discriminator's error rate.

A known dataset serves as the initial training data for the discriminator. Training involves presenting it with samples from the training dataset until it achieves acceptable accuracy. The generator is trained based on whether it succeeds in fooling the discriminator. Typically, the generator is seeded with randomized input that is sampled from a predefined latent space (e.g. a multivariate normal distribution). Thereafter, candidates synthesized by the generator are evaluated by the discriminator. Independent backpropagation procedures are applied to both networks so that the generator produces better samples, while the discriminator becomes more skilled at flagging synthetic samples. When used for image generation, the generator is typically a deconvolutional neural network, and the discriminator is a convolutional neural network.

Relation to other statistical machine learning methods

GANs are implicit generative models, which means that they do not explicitly model the likelihood function nor provide a means for finding the latent variable corresponding to a given sample, unlike alternatives such as flow-based generative model.

Compared to fully visible belief networks such as WaveNet and PixelRNN and autoregressive models in general, GANs can generate one complete sample in one pass, rather than multiple passes through the network.

Compared to Boltzmann machines and linear ICA, there is no restriction on the type of function used by the network.

Since neural networks are universal approximators, GANs are asymptotically consistent. As of 2026, variational autoencoders have been proven to be universal approximators.

Mathematical properties

Measure-theoretic considerations

This section provides some of the mathematical theory behind these methods.

In modern probability theory based on measure theory, a probability space also needs to be equipped with a σ -algebra. As a result, a more rigorous definition of the GAN game would make the following changes: Each probability space

(

Ω

,

B

,

μ

ref

)

$$\{\Omega, \mathcal{B}, \mu_{\text{ref}}\}$$

defines a GAN game.

The generator's strategy set is

$\mathcal{P}$

(

Ω

,

B

)

$$\{\mathcal{P}(\Omega, \mathcal{B})\}$$

, the set of all probability measures

μ

G

$$\mu_G$$

on the measure-space

(

Ω

,

$\mathcal{B}$

)

$$\{\Omega, \mathcal{B}\}$$

.

The discriminator's strategy set is the set of Markov kernels

μ

$\mathcal{D}$

:

(

Ω

,

B

)

$\rightarrow$

P

(

[

0

,

1

]

,

B

(
[
0
,
1
]
)
)

$$\mu_{D}:(\Omega ,\mathcal{B})\rightarrow \mathcal{P}([0,1],\mathcal{B}([0,1]))$$

, where

B

(
[

0
,
1
]
)

$\{\mathrm{B}\}([0,1])$

is the Borel σ -algebra on

[
0
,
1
]

$[0,1]$

.Since issues of measurability never arise in practice, these will not concern us further.

Choice of the strategy set

In the most generic version of the GAN game described above, the strategy set for the

discriminator contains all Markov kernels

μ

D

:

Ω

$\rightarrow$

P

[

0

,

1

]

$$\{\mu_{D:\Omega} \rightarrow \mathcal{P}[0,1]\}$$

, and the strategy set for the generator contains arbitrary probability distributions

$$\mu$$

$$G$$

$$\{\displaystyle \mu _{G}\}$$

on

$$\Omega$$

$$\{\displaystyle \Omega \}$$

.

However, as shown below, the optimal discriminator strategy against any

μ

G

$$\mu_G$$

is deterministic, so there is no loss of generality in restricting the discriminator's strategies to deterministic functions

D

:

Ω

$\rightarrow$

[

0

,

1

]

$$\{\displaystyle D:\Omega \rightarrow [0,1]\}$$

. In most applications,

D

$$\{\displaystyle D\}$$

is a deep neural network function.

As for the generator, while

μ

G

$$\mu_G$$

could theoretically be any computable probability distribution, in practice, it is usually implemented as a pushforward:

$$\mu$$

$$G$$

$$=$$

$$\mu$$

$$Z$$

$$\circ$$

$$G$$

$$-$$

$$\mu_G = \mu_Z \circ G^{-1}$$

. That is, start with a random variable

Z

$\sim$

μ

Z

$$Z \sim \mu_Z$$

, where

μ

Z

$$\{\displaystyle \mu _{Z}\}$$

is a probability distribution that is easy to compute (such as the uniform distribution, or the Gaussian distribution), then define a function

G

:

Ω

Z

$\rightarrow$

Ω

$$G:\Omega_Z\rightarrow \Omega }$$

. Then the distribution

$$\mu$$

$$G$$

$$\mu_G$$

is the distribution of

$$G$$

$$($$

$$z$$

$$)$$

$$G(z)$$

.

Consequently, the generator's strategy is usually defined as just

$$G$$

$$G$$

, leaving

$$z$$

$$\sim$$

$$\mu$$

$$z$$

$$z \sim \mu_{\{Z\}}$$

implicit. In this formalism, the GAN game objective is

L

$($

G

$,$

D

$)$

$:=$

E

x

$\sim$

μ

ref

[
ln

D
(
x
)
]
+

E

z
~

μ

z

[
ln

(
1
-
D
(
G
(
z
)
)
)
]
.

$$\{ \displaystyle L(G,D) := \operatornamename {E} _{x \sim \mu _{\text{ref}}} [\ln D(x)] + \operatornamename {E} _{z \sim \mu _Z} [\ln (1 - D(G(z)))] . \}$$

Generative reparametrization

The GAN architecture has two main components. One is casting optimization into a game, of form

min

G

max

D

L

(

G

,

D

)

$$\min _G \max _D L(G, D)$$

, which is different from the usual kind of optimization, of form

min

θ

L

(

θ

)

$$\min_{\theta} L(\theta)$$

. The other is the decomposition of

μ

G

$$\mu_G$$

into

$$\mu$$

$$Z$$

$$\circ$$

$$G$$

$$-$$

$$1$$

$$\mu_{Z\circ G^{-1}}$$

, which can be understood as a reparametrization trick.

To see its significance, one must compare GAN with previous methods for learning generative models, which were plagued with "intractable probabilistic computations that arise in maximum likelihood estimation and related strategies".

At the same time, Kingma and Welling and Rezende et al. developed the same idea of reparametrization into a general stochastic backpropagation method. Among its first applications was the variational autoencoder.

Move order and strategic equilibria

In the original paper, as well as most subsequent papers, it is usually assumed that the generator moves first, and the discriminator moves second, thus giving the following minimax game:

min

μ

G

max

μ

D

L

(

μ

G

,

μ

D

)

:=

E

x

~

μ

ref

,

y

~

μ

D

(

x

)

[

ln

y

]

+

E

x

~

μ

G

,

y

~

μ

D

(
x
)

[
ln

(
1
-
y
)
]
.

$$\min_{\mu_G} \max_{\mu_D} L(\mu_G, \mu_D) = \operatorname{E}_{x \sim \mu_{\text{ref}}, y \sim \mu_D(x)} [\ln y] + \operatorname{E}_{x \sim \mu_G, y \sim \mu_D(x)} [\ln(1-y)].$$

If both the generator's and the discriminator's strategy sets are spanned by a finite number of strategies, then by the minimax theorem,

min

μ

G

max

μ

D

L

(

μ

G

,

μ

D

)

=

max

μ

D

min

μ

G

L

(

μ

G

,

μ

D

)

$$\min_{\mu_G} \max_{\mu_D} L(\mu_G, \mu_D) = \max_{\mu_D} \min_{\mu_G} L(\mu_G, \mu_D)$$

that is, the move order does not matter.

However, since the strategy sets are both not finitely spanned, the minimax theorem does not apply, and the idea of an "equilibrium" becomes delicate. To wit, there are the following different concepts of equilibrium:

Equilibrium when generator moves first, and discriminator moves second:

μ

$\wedge$

G

∈

arg

min

μ

G

max

μ

D

L

(

μ

G

,

μ

D

)

,

μ

$\wedge$

D

∈

arg

max

μ

D

L

(

μ

$\wedge$

G

,

μ

D

)

,

$$\{\displaystyle \hat{\mu}_{G}\in \arg \min_{\mu_{G}}\max_{\mu_{D}}L(\mu_{G},\mu_{D}),\quad \hat{\mu}_{D}\in \arg \max_{\mu_{D}}L(\hat{\mu}_{G},\mu_{D}),\quad \}$$

Equilibrium when discriminator moves first, and generator moves second:

$$\mu^{\wedge}$$

$$D$$

$$\in \arg$$

$$\max$$

$$\mu$$

D

min

μ

G

L

(

μ

G

,

μ

D

)

,

μ

$\wedge$

G

$\in$

arg

min

μ

G

L

(

μ

G

,

μ

^

D

)

,

$$\{\hat{\mu}_D \in \arg \max_{\mu_D} \min_{\mu_G} L(\mu_G, \mu_D), \hat{\mu}_G \in \arg \min_{\mu_G} L(\mu_G, \hat{\mu}_D)\}$$

Nash equilibrium

(

$$\mu^{\wedge}$$

D

,

μ

$\wedge$

G

)

$$\{\hat{\mu}_D, \hat{\mu}_G\}$$

, which is stable under simultaneous move order:

μ

$\wedge$

D

$\in$

arg

max

μ

D

L

(

μ

$\wedge$

G

,

μ

D

)

,

μ

$\wedge$

G

$\in$

arg

min

μ

G

L

(

μ

G

,

μ

$\wedge$

D

)

$$\hat{\mu}_D \in \arg \max_{\mu_D} L(\hat{\mu}_G, \mu_D), \quad \hat{\mu}_G \in \arg \min_{\mu_G} L(\mu_G, \hat{\mu}_D)$$

For general games, these equilibria do not have to agree, or even to exist. For the original GAN game, these equilibria all exist, and are all equal. However, for more general GAN games, these do not necessarily exist, or agree.

Main theorems for GAN game

The original GAN paper proved the density-based optimal-discriminator formula and global minimax optimum. A measure-theoretic treatment gives the same binary adversarial calculation using Radon–Nikodym derivatives.

Interpretation: For any fixed generator strategy

μ

G

$$\mu_G$$

, the optimal discriminator keeps track of the likelihood ratio between the reference distribution and the generator distribution. Since

$$D$$

$*$

$$=$$

$$\rho$$

$$\text{ref}$$

$$D^*=\rho_{\text{ref}}$$

and

1

-

D

*

=

ρ

G

$$1-D^{\ast}=\rho _G$$

,

D

*

(

x

)

1

-

D

*

(

x

)

=

ρ

ref

(

x

)

ρ

G

(

x

)

μ

-almost everywhere on

{

ρ

G

>

0

}

.

$$\frac{D^{\ast}(x)}{1-D^{\ast}(x)}=\frac{\rho_{\text{\textit{ref}}}(x)}{\rho_G(x)}\quad$$

$$\mu\{\text{-almost everywhere on }\}\{\rho_G>0\}.$$

On

{

ρ

G

=

0

}

$$\{\rho_G=0\}$$

,

D

*

$$=1$$

$${\displaystyle D^{\ast}=1}$$

; on

{

ρ

ref

$$=0\}$$

$${\displaystyle \backslash\rho_{\text{ref}}=0\}}$$

,

D

*

=

0

$$D^*=0$$

, in each case

μ

$$\mu$$

-almost everywhere. If

μ

ref

<<

μ

G

$$\frac{\mu_{\text{ref}}}{\mu_G}$$

, the displayed ratio is a version of

d

μ

ref

/

d

μ

G

$$\frac{d\mu_{\text{ref}}}{d\mu_G}$$

μ

G

$$\mu_G$$

-almost everywhere. Where both densities are positive,

$$D$$

$*$

$$($$

$$x$$

$$)$$

$$=$$

$$\sigma$$

$$($$

$$\ln$$

ρ

ref

(

x

)

-

ln

ρ

G

(

x

)

)

μ

-almost everywhere

,

$$D^{\ast}(x)=\sigma \left(\ln \rho _{\text{ref}}(x)-\ln \rho _{G}(x)\right)\quad \mu$$

 $\{\text{-almost everywhere}\},\}$

where

σ

$$\sigma$$

is the logistic function.

In the equal-prior mixture model

M

=

(

μ

ref

+

μ

G

)

/

2

$$M=(\mu_{\text{ref}}+\mu_{\text{G}})/2$$

,

D

*

$$D^{\ast}$$

is a version of the posterior probability that an observation came from the reference distribution,

M

$$M$$

-almost everywhere.

Training and evaluating GAN

Training

Unstable convergence

While the GAN game has a unique global equilibrium point when both the generator and discriminator have access to their entire strategy sets, the equilibrium is no longer guaranteed when they have a restricted strategy set.

In practice, the generator has access only to measures of form

μ

z

$\circ$

G

θ

-

1

$$\mu_Z \circ G_{\theta}^{-1}$$

, where

$$G$$

$$\theta$$

$$G_{\theta}$$

is a function computed by a neural network with parameters

$$\theta$$

$$\theta$$

, and

μ

Z

$$\mu_Z$$

is an easily sampled distribution, such as the uniform or normal distribution. Similarly, the discriminator has access only to functions of form

D

ζ

$$D_{\zeta}$$

, a function computed by a neural network with parameters

ζ

$$\zeta$$

. These restricted strategy sets take up a vanishingly small proportion of their entire strategy sets.

Further, even if an equilibrium still exists, it can only be found by searching in the high-dimensional space of all possible neural network functions. The standard strategy of using gradient descent to find the equilibrium often does not work for GAN, and often the game "collapses" into one of several failure modes. To improve the convergence stability, some training strategies start with an easier task, such as generating low-resolution images or simple images (one object with uniform background), and gradually increase the difficulty of the task during training. This essentially translates to applying a curriculum learning scheme.

Mode collapse

GANs often suffer from mode collapse where they fail to generalize properly, missing entire modes from the input data. For example, a GAN trained on the MNIST dataset containing many samples of each digit might only generate pictures of digit 0. This was termed "the

Helvetica scenario".

A typical mechanism for mode collapse is the generator only generating one or a few of the likely values, or a very incomplete picture of the target distribution. As the discriminator is only trained to distinguish real from fake samples, it will correctly identify the generated samples as real, but no penalty is imposed on the GAN's ability to generate data that represents the full range of the target distribution.

Weak discriminators, for instance underparametrized ones, or ones trained too slow compared to the generator, may as well be unable to fully discriminate over the entire support of the distribution, and only become able to correctly discriminate a very incomplete part of the target distribution.

Some researchers perceive the root problem to be a weak discriminative network that fails to notice the pattern of omission, while others assign blame to a bad choice of objective function. Many solutions have been proposed, but it is still an open problem.

Even the state-of-the-art architecture, BigGAN (2019), could not avoid mode collapse. The authors resorted to "allowing collapse to occur at the later stages of training, by which time a model is sufficiently trained to achieve good results".

Two time-scale update rule

The two time-scale update rule (TTUR) is proposed to make GAN convergence more stable by making the learning rate of the generator lower than that of the discriminator. They prove that when trained this way, GANs "converge, under mild assumptions to a stationary local Nash equilibrium". They further show that this property extends to the use of the Adam optimizer, which is commonly used in stochastic gradient descent.

A local Nash equilibrium in no way signifies an absence of mode collapse - for instance, a GAN trained on MNIST collapsed to generating a single digit may satisfy the hypotheses of the paper, while still presenting mode collapse.

Vanishing gradient

Conversely, if the discriminator learns too fast compared to the generator, then the discriminator could almost perfectly distinguish

μ

G

θ

,

μ

ref

$$\mu_{G_{\theta}}, \mu_{\text{ref}}$$

. In such case, the generator

$$G$$

$$\theta$$

$$G_{\theta}$$

could be stuck with a very high loss no matter which direction it changes its

$$\theta$$

$$\theta$$

, meaning that the gradient

∇

θ

L

$($

G

θ

$,$

D

ζ

$)$

$$\nabla_{\theta} L(G_{\theta}, D_{\zeta})$$

would be close to zero. In such case, the generator cannot learn, a case of the vanishing gradient problem.

Intuitively speaking, the discriminator is too good, and since the generator cannot take any small step (only small steps are considered in gradient descent) to improve its payoff, it does not even try.

One important method for solving this problem is the Wasserstein GAN.

Evaluation

GANs are usually evaluated by Inception score (IS), which measures how varied the generator's outputs are (as classified by an image classifier, usually Inception-v3), or Fréchet inception distance (FID), which measures how similar the generator's outputs are to a reference set (as classified by a learned image featurizer, such as Inception-v3 without its final layer). Many papers that propose new GAN architectures for image generation report how their architectures break the state of the art on FID or IS.

Another evaluation method is the Learned Perceptual Image Patch Similarity (LPIPS), which starts with a learned image featurizer

θ

:

Image

$\rightarrow$

$\mathbb{R}$

n

$$f_{\theta}:\text{Image}\rightarrow \mathbb{R}^n$$

, and finetunes it by supervised learning on a set of

(

x

,

x

,

,

p

e

r

c

e

p

t

u

a

l

d

i

f

f

e

r

e

n

c

e

(

x

,

x

,

)

)

$$\{ \operatorname{perceptual-difference}(x,x') \}$$

, where

x

$$x$$

is an image,

x
,

$\{\displaystyle x'\}$

is a perturbed version of it, and

p
e
r
c
e
p
t
u
a

l

d

i

f

f

e

r

e

n

c

e

(

x

,

x

,

)

$$\operatorname{perceptual-difference}(x,x')$$

is how much they differ, as reported by human subjects. The model is finetuned so that it can approximate

$\|$

f

θ

$($

x

$)$

$-$

f

θ

$($

x

$,$

)

||

≈

p

e

r

c

e

p

t

u

a

l

d

i

f

f

e

r

e

n

c

e

(

x

,

x

,

)

$$\|f_{\theta}(x)-f_{\theta}(x')\| \approx \text{perceptual-difference}(x,x')$$

. This finetuned model is then used to define

LPIPS

(

x

,

x

,

)

:=

||

f

θ

(

x

)

-

f

θ

(

x

,

)

||

$$\operatorname{LPIPS}(x,x'):=\|f_{\theta}(x)-f_{\theta}(x')\|$$

.

Other evaluation methods are reviewed in.

Variants

There is a veritable zoo of GAN variants. Some of the most prominent are as follows:

Conditional GAN

Conditional GANs are similar to standard GANs except they allow the model to conditionally generate samples based on additional information. For example, if we want to generate a cat face given a dog picture, we could use a conditional GAN.

The generator in a GAN game generates

μ

G

$$\mu_G$$

, a probability distribution on the probability space

$$\Omega$$

$$\Omega$$

. This leads to the idea of a conditional GAN, where instead of generating one probability distribution on

$$\Omega$$

$$\Omega$$

, the generator generates a different probability distribution

μ

G

(

c

)

$$\mu_{G}(c)$$

on

Ω

$$\Omega$$

, for each given class label

c

$$c$$

.

For example, for generating images that look like ImageNet, the generator should be able to generate a picture of cat when given the class label "cat".

In the original paper, the authors noted that GAN can be trivially extended to conditional GAN by providing the labels to both the generator and the discriminator.

Concretely, the conditional GAN game is just the GAN game with class labels provided:

L

(

μ

G

,

D

)

:=

E

c

~

μ

C

,

x

~

μ

ref

(

c

)

[

ln

D

(

x

,

c

)

]

+

E

c

~

μ

C

,

x

~

μ

G

(

c

)

[

ln

(

1

-

D

(

x

,

c

)

)

]

$$L(\mu_G,D):=\operatorname{E}_{c\sim \mu_C,x\sim \mu_{\text{ref}}(c)}[\ln D(x,c)]+\operatorname{E}_{c\sim \mu_C,x\sim \mu_G(c)}[\ln(1-D(x,c))]$$

where

$$\mu$$

$$C$$

$$\mu_C$$

is a probability distribution over classes,

$$\mu$$

ref

(
c
)

$$\mu_{\text{ref}}(c)$$

is the probability distribution of real images of class

c

$$c$$

, and

μ

G

(
c
)

$$\mu_{G(c)}$$

the probability distribution of images generated by the generator when given class label

c

$$c$$

.

In 2017, a conditional GAN learned to generate 1000 image classes of ImageNet.

GANs with alternative architectures

The GAN game is a general framework and can be run with any reasonable parametrization of the generator

G

$$G$$

and discriminator

D

$$D$$

. In the original paper, the authors demonstrated it using multilayer perceptron networks and convolutional neural networks. Many alternative architectures have been tried.

Deep convolutional GAN (DCGAN): For both generator and discriminator, uses only deep networks consisting entirely of convolution-deconvolution layers, that is, fully convolutional networks.

Self-attention GAN (SAGAN): Starts with the DCGAN, then adds residually-connected standard self-attention modules to the generator and discriminator.

Variational autoencoder GAN (VAEGAN): Uses a variational autoencoder (VAE) for the generator.

Transformer GAN (TransGAN): Uses the pure transformer architecture for both the generator

and discriminator, entirely devoid of convolution-deconvolution layers.

Flow-GAN: Uses flow-based generative model for the generator, allowing efficient computation of the likelihood function.

GANs with alternative objectives

Many GAN variants are merely obtained by changing the loss functions for the generator and discriminator.

Original GAN:

We recast the original GAN objective into a form more convenient for comparison:

{

min

D

L

D

(

D

,

μ

G

)

=

-

E

x

~

μ

G

[

ln

D

(

x

)

]

-

E

x

~

μ

ref

[

ln

(

1

-

D

(

x

)

)

]

min

G

L

G

(

D

,

μ

G

)

=

-

E

x

~

μ

G

[
ln

(
1
-
D
(
x
)
)
]

$$\{\displaystyle \begin{cases} \min_{D} L_{D}(D, \mu_{G}) = -\operatorname{E}_{x \sim \mu_{G}} [\ln D(x)] - \operatorname{E}_{x \sim \mu_{\text{ref}}} [\ln(1-D(x))] \\ \min_{G} L_{G}(D, \mu_{G}) = -\operatorname{E}_{x \sim \mu_{G}} [\ln(1-D(x))] \end{cases}$$

Original GAN, non-saturating loss:

This objective for generator was recommended in the original paper for faster convergence.

L

G

=

E

x

~

μ

G

$$[\ln D(x)]$$

$$\mathbb{E}_{x \sim \mu_G} [\ln D(x)]$$

The effect of using this objective is analyzed in Section 2.2.2 of Arjovsky et al.

Original GAN, maximum likelihood:

$$L$$

$$G$$

=

E

x

~

μ

G

[

(

exp

◦

σ

-

1

°

D

)

(

x

)

]

$$\mathbb{E}_{x \sim \mu_G} [\exp(\sigma^{-1} \text{circ } D(x))]$$

where

σ

$$\sigma$$

is the logistic function. When the discriminator is optimal, the generator gradient is the same as in maximum likelihood estimation, even though GAN cannot perform maximum likelihood estimation itself.

Hinge loss GAN:

L

D

=

-

E

x

~

p

ref

[

min

(

0

,

-

1

+

D

(

x

)

)

]

-

E

x

~

μ

G

[

min

(

0

,

-

1

-

D

(

x

)

)

]

$$\begin{aligned} & \{\displaystyle L_{\{D\}}=-\operatornamename{E}\}_{x\sim p_{\{\text{ref}\}}}\left[\min \right. \\ & \left.\left(0,-1+D(x)\right)\right]-\operatornamename{E}\}_{x\sim \mu_{\{G\}}}\left[\min \right. \\ & \left.\left(0,-1-D\left(x\right)\right)\right]\} \end{aligned}$$

L

G

=

-

E

x

~

μ

G

[

D

(

x

)

]

$$L_G = -\operatorname{E}_{x \sim \mu_G} [D(x)]$$

Least squares GAN:

L

D

=

E

x

~

μ

ref

[

(

D

(

x

)

-

b

)

2

]

+

E

x

~

μ

G

[
 (
 D
 (
 x
)
 -
 a

)

 2

]

$$\frac{L_{\{D\}}=\operatornamename{E}_{\{x\sim \mu_{\{\text{ref}\}}[(D(x)-b)^2]}+\operatornamename{E}_{\{x\sim \mu_{\{G\}}[(D(x)-a)^2]}}$$

L

G

=

E

x

~

μ

G

[

(

D

(

x

)

-

c

)

2

]

$$L_{\{G\}}=\operatornamename{E}_{\{x\sim \mu_{\{G\}}\}}[(D(x)-c)^{2}]$$

where

a

,

b

,

c

$$a,b,c$$

are parameters to be chosen. The authors recommended

a

$=$

$-$

1

$,$

b

$=$

1

$,$

c

$=$

0

$$\{a=-1, b=1, c=0\}$$

.

Wasserstein GAN (WGAN)

The Wasserstein GAN modifies the GAN game at two points:

The discriminator's strategy set is the set of measurable functions of type

D

:

Ω

$\rightarrow$

$\mathbb{R}$

$$\{D:\Omega \rightarrow \mathbb{R}\}$$

with bounded Lipschitz norm:

$\|$

D

$\|$

L

≤

K

$$\|D\|_L \leq K$$

, where

K

$$K$$

is a fixed positive constant.

The objective is

L

W

G

A

N

(

μ

G

,

D

)

:=

E

x

~

μ

G

[
D
(
x
)
]
-

E

x

~

μ

ref

[

D

(

x

)

]

$$\{\displaystyle L_{\text{WGAN}}(\mu_G, D) := \operatorname{E}_{x \sim \mu_G} [D(x)] - \mathbb{E}_{x \sim \mu_{\text{ref}}} [D(x)]\}$$

One of its purposes is to solve the problem of mode collapse (see above). The authors claim "In no experiment did we see evidence of mode collapse for the WGAN algorithm".

GANs with more than two players

Adversarial autoencoder

An adversarial autoencoder (AAE) is more autoencoder than GAN. The idea is to start with a plain autoencoder, but train a discriminator to discriminate the latent vectors from a reference distribution (often the normal distribution).

InfoGAN

In conditional GAN, the generator receives both a noise vector

z

$\{\backslashdisplaystyle z\}$

and a label

c

$\{\backslashdisplaystyle c\}$

, and produces an image

G

(

z

,

c

)

$\{\backslashdisplaystyle G(z,c)\}$

. The discriminator receives image-label pairs

(
x
,
c
)

$$\{ \displaystyle (x,c) \}$$

, and computes

D
(
x
,
c
)

$$D(x,c)$$

.

When the training dataset is unlabeled, conditional GAN does not work directly.

The idea of InfoGAN is to decree that every latent vector in the latent space can be decomposed as

$$(z, c)$$

$$(z, c)$$

z : an incompressible noise part

$$z$$

$$z$$

, and an informative label part

c

$${\displaystyle c}$$

, and encourage the generator to comply with the decree, by encouraging it to maximize

I

(

c

,

G

(

z

,

c

)

)

$$I(c,G(z,c))$$

, the mutual information between

$$c$$

$$c$$

and

$$G$$

$$($$

$$z$$

$$,$$

$$c$$

$$)$$

$$G(z,c)$$

, while making no demands on the mutual information

z

$$z$$

between

G

(

z

,

c

)

$$G(z,c)$$

.

Unfortunately,

I

(

c

,

G

(

z

,

c

)

)

$$\{ \displaystyle I(c,G(z,c)) \}$$

is intractable in general, The key idea of InfoGAN is Variational Mutual Information Maximization: indirectly maximize it by maximizing a lower bound

I

^

(

G

,

Q

)

=

E

Z

~

μ

Z

,

C

~

μ

c

[

ln

Q

(

c

|

G

(

z

,

c

)

)

]

;

I

$$(c, G(z, c)) \geq$$

$$\sup$$

$$Q$$

$$I^{\wedge}$$

$$($$

G

,

Q

)

$$\hat{I}(G,Q)=\mathbb{E}_{z\sim \mu_Z,c\sim \mu_C}[\ln Q(c\mid G(z,c))];\quad I(c,G(z,c))\geq \sup_Q\{\hat{I}(G,Q)\}$$

where

Q

$$Q$$

ranges over all Markov kernels of type

Q

:

Ω

Y

→

P

(

Ω

C

)

$$Q:\Omega_Y\rightarrow \mathcal{P}(\Omega_C)$$

.

The InfoGAN game is defined as follows: Three probability spaces define an InfoGAN game:

(

Ω

X

,

μ

ref

)

$$\{\Omega_X, \mu_{\text{ref}}\}$$

, the space of reference images.

(

Ω

Z

,

μ

Z

)

$$\{\Omega_Z, \mu_Z\}$$

, the fixed random noise generator.

(

Ω

C

,

μ

C

)

$$\{\Omega_C, \mu_C\}$$

, the fixed random information generator.

There are 3 players in 2 teams: generator, Q, and discriminator. The generator and Q are on one team, and the discriminator on the other team.

The objective function is

L

(
G
,
Q
,
D
)
=

L

G
A
N

(
G
,
D
)
-
 λ

I

^

(

G

,

Q

)

$$L(G,Q,D)=L_{\{GAN\}}(G,D)-\lambda \hat{I}(G,Q)$$

where

L

G

A

N

(
G
,
D
)
=

E

x
~

μ

ref

,

[
ln
D
(

x

)

]

+

E

z

~

μ

z

[

ln

(

1

-

D

(

G
(
z
,
c
)
)
)
)
]

$$L_{\text{GAN}}(G,D)=\operatorname{E}_{x\sim \mu_{\text{ref}}}\left[\ln D(x)\right]+\operatorname{E}_{z\sim \mu_Z}\left[\ln(1-D(G(z,c)))\right]$$

is the original GAN game objective, and

I
^

(
G
,
Q
)
=

E

Z
~

μ

Z

,
C
~

μ

C

[
ln

Q
(
c
|
G
(
z
,
c
)
)
]

$$\hat{I}(G,Q)=\mathbb{E}_{z\sim \mu_Z,c\sim \mu_C}[\ln Q(c|G(z,c))]$$

Generator-Q team aims to minimize the objective, and discriminator aims to maximize it:

min

G

,

Q

max

D

L

(

G

,

Q

,

D

)

$$\min_{G,Q}\max_{D}L(G,Q,D)$$

Bidirectional GAN (BiGAN)

The standard GAN generator is a function of type

G

:

Ω

Z

→

Ω

X

$$G:\Omega_Z\rightarrow \Omega_X$$

, that is, it is a mapping from a latent space

$$\Omega$$

$$Z$$

$$\{\displaystyle \Omega _{Z}\}$$

to the image space

$$\Omega$$

$$X$$

$$\Omega_X$$

. This can be understood as a "decoding" process, whereby every latent vector

$$z$$

$$\in$$

$$\Omega$$

$$Z$$

$$z\in \Omega_Z$$

is a code for an image

$$x$$

$$\in$$

Ω

X

$$\{x \in \Omega_X\}$$

, and the generator performs the decoding. This naturally leads to the idea of training another network that performs "encoding", creating an autoencoder out of the encoder-generator pair.

Already in the original paper, the authors noted that "Learned approximate inference can be performed by training an auxiliary network to predict

z

$$\{z\}$$

given

x

$\{\displaystyle x\}$

". The bidirectional GAN architecture performs exactly this.

The BiGAN is defined as follows: Two probability spaces define a BiGAN game:

(

Ω

X

,

μ

X

)

$$(\Omega_X, \mu_X)$$

, the space of reference images.

(

Ω

Z

,

μ

Z

)

$$(\Omega_Z, \mu_Z)$$

, the latent space.

There are 3 players in 2 teams: generator, encoder, and discriminator. The generator and encoder are on one team, and the discriminator on the other team.

The generator's strategies are functions

G

:

Ω

Z

→

Ω

X

$$G:\Omega _{Z}\rightarrow \Omega _{X}$$

, and the encoder's strategies are functions

E

:

Ω

X

$\rightarrow$

Ω

Z

$$E:\Omega _{X}\rightarrow \Omega _{Z}$$

. The discriminator's strategies are functions

D

:

Ω

X

$\rightarrow$

[

0

,

1

]

$$D:\Omega_X\rightarrow [0,1]$$

.

The objective function is

L

(

G

,

E

,

D

)

=

E

x

~

μ

x

[

ln

D

(

x

,

E

(

x

)

)

]

+

E

Z

~

μ

Z

[
ln

(
1
-
D
(
G
(
z
)
,
z
)
)
]

$$\{L(G,E,D)=\mathbb{E}_{x\sim \mu_X}[\ln D(x,E(x))]+\mathbb{E}_{z\sim \mu_Z}[\ln(1-D(G(z),z))]\}$$

Generator-encoder team aims to minimize the objective, and discriminator aims to maximize

it:

min

G

,

E

max

D

L

(

G

,

E

,

D

)

$$\min_{G,E} \max_D L(G,E,D)$$

In the paper, they gave a more abstract definition of the objective as:

L

$($

G

$,$

E

$,$

D

$)$

$=$

E

$($

x

$,$

z

)

~

μ

E

,

X

[

ln

D

(

x

,

z

)

]

+

E

(

x

,

z

)

~

μ

G

,

Z

[

ln

(

1

-

D

(

x

,

z

)

)

]

$$L(G,E,D)=\mathbb{E}_{(x,z)\sim \mu_{E,X}}[\ln D(x,z)]+\mathbb{E}_{(x,z)\sim \mu_{G,Z}}[\ln(1-D(x,z))]$$

where

μ

E

,

X

(

d

x

,

d

z

)

=

μ

X

(

d

x

)

.

δ

E

(

x

)

(

d
z
)

$$\mu_{\{E,X\}}(dx,dz)=\mu_X(dx)\cdot \delta_{\{E(x)\}}(dz)$$

is the probability distribution on

Ω

X

$\times$

Ω

Z

$$\{\displaystyle \Omega_{\{X\}}\times \Omega_{\{Z\}}\}$$

obtained by pushing

$$\mu$$

$$X$$

$$\{\displaystyle \mu_{\{X\}}\}$$

forward via

$$x$$

$$\mapsto$$

$$($$

$$x$$

$$,$$

$$E$$

(
x
)
)

$$x \mapsto (x, E(x))$$

, and

μ

G

,

Z

(
d
x
,
d
z

)

=

δ

G

(

z

)

(

d

x

)

.

μ

Z

(

d

z

)

$$\mu_{G,Z}(dx,dz)=\delta_{G(z)}(dx)\cdot \mu_Z(dz)$$

is the probability distribution on

$$\Omega$$

$$X$$

$$\times$$

$$\Omega$$

$$Z$$

$$\Omega_X\times \Omega_Z$$

obtained by pushing

μ

Z

$$\mu_Z$$

forward via

Z

↦

(

G

(

x

)

,

Z

)

$$\{ \displaystyle z \mapsto (G(x), z) \}$$

.

Applications of bidirectional models include semi-supervised learning, interpretable machine learning, and neural machine translation.

CycleGAN

CycleGAN is an architecture for performing translations between two domains, such as between photos of horses and photos of zebras, or photos of night cities and photos of day cities.

The CycleGAN game is defined as follows: There are two probability spaces

(

Ω

X

,

μ

X

)

,

(

Ω

Y

,

μ

Y

)

$$\{\backslash displaystyle (\backslash Omega _{X},\backslash mu _{X}),(\backslash Omega _{Y},\backslash mu _{Y})\}$$

, corresponding to the two domains needed for translations fore-and-back.

There are 4 players in 2 teams: generators

G

X

:

Ω

X

$\rightarrow$

Ω

Y

,

G

Y

:

Ω

Y

$\rightarrow$

Ω

X

$$\{ \displaystyle G_X:\Omega_X\to \Omega_Y,G_Y:\Omega_Y\to \Omega_X \}$$

, and discriminators

D

X

:

Ω

X

$\rightarrow$

[

0

,

1

]

,

D

Y

:

Ω

Y

$\rightarrow$

[

0

,

1

]

$$\{D_X:\Omega_X\rightarrow [0,1],D_Y:\Omega_Y\rightarrow [0,1]\}$$

.

The objective function is

L

(

G

X

,

G

Y

,

D

X

,

D

Y

)

=

L

G

A

N

(

G

X

,

D

X

)

+

L

G

A

N

(

G

Y

,

D

Y

)

+

λ

L

c

y

c

l

e

(

G

X

,

G

Y

)

$\{\displaystyle$

$$L(G_{\{X\}},G_{\{Y\}},D_{\{X\}},D_{\{Y\}})=L_{\{GAN\}}(G_{\{X\}},D_{\{X\}})+L_{\{GAN\}}(G_{\{Y\}},D_{\{Y\}})+\backslash\lambda$$
$$L_{\{cycle\}}(G_{\{X\}},G_{\{Y\}})\}$$

where

$$\lambda$$

$$\backslash\displaystyle \backslash\lambda \}$$

is a positive adjustable parameter,

L

G

A

N

$$\mathcal{L}_{\text{GAN}}$$

is the GAN game objective, and

$\mathcal{L}$

c

y

c

l

e

$$\mathcal{L}_{\text{cycle}}$$

is the cycle consistency loss:

L

c

y

c

l

e

(

G

X

,

G

Y

)

=

E

x

~

μ

x

||

G

x

(

G

Y

(

x

)

)

-

x

||

+

E

y

~

μ

Y

||

G

Y

(

G

X

(

y

)

)

-

y

||

$$\{\displaystyle L_{\text{cycle}}(G_{\{X\}},G_{\{Y\}})=E_{\{x\sim \mu_{\{X\}}\}}\|G_{\{X\}}(G_{\{Y\}}(x))-x\|+E_{\{y\sim \mu_{\{Y\}}\}}\|G_{\{Y\}}(G_{\{X\}}(y))-y\|\}$$

The generators aim to minimize the objective, and the discriminators aim to maximize it:

min

G

X

,

G

Y

max

D

X

,

D

Y

L

(

G

X

,

G

Y

,

D

X

,

D

Y

)

$$\{\displaystyle \min _{G_{\{X\}},G_{\{Y\}}}\backslash \max _{D_{\{X\}},D_{\{Y\}}}L(G_{\{X\}},G_{\{Y\}},D_{\{X\}},D_{\{Y\}})\}$$

Unlike previous work like pix2pix, which requires paired training data, cycleGAN requires no paired data. For example, to train a pix2pix model to turn a summer scenery photo to winter scenery photo and back, the dataset must contain pairs of the same place in summer and winter, shot at the same angle; cycleGAN would only need a set of summer scenery photos, and an unrelated set of winter scenery photos.

GANs with particularly large or small scales

BigGAN

The BigGAN is essentially a self-attention GAN trained on a large scale (up to 80 million parameters) to generate large images of ImageNet (up to 512 x 512 resolution), with numerous engineering tricks to make it converge.

Invertible data augmentation

When there is insufficient training data, the reference distribution

μ

ref

$$\mu_{\text{ref}}$$

cannot be well-approximated by the empirical distribution given by the training dataset. In such cases, data augmentation can be applied, to allow training GAN on smaller datasets. Naïve data augmentation, however, brings its problems.

Consider the original GAN game, slightly reformulated as follows:

{

min

D

L

D

(

D

,

μ

G

)

=

-

E

x

~

μ

ref

[

ln

D

(

x

)

]

-

E

x

~

μ

G

[

ln

(

1

-

D

(

x

)

)

]

min

G

L

G

(

D

,

μ

G

)

=

-

E

x

~

μ

G

[

ln

(

1

-

D

(
x
)
)
]

$$\{\displaystyle \begin{cases} \min_{D} L_D(D, \mu_G) = -\operatorname{E}_{x \sim \mu_{\text{ref}}} [\ln D(x)] - \operatorname{E}_{x \sim \mu_G} [\ln(1-D(x))] \\ \min_{G} L_G(D, \mu_G) = -\operatorname{E}_{x \sim \mu_G} [\ln(1-D(x))] \end{cases}$$

Now we use data augmentation by randomly sampling semantic-preserving transforms

T
:
Ω
→
Ω

$$T:\Omega \rightarrow \Omega$$

and applying them to the dataset, to obtain the reformulated GAN game:

{

min

D

L

D

(

D

,

μ

G

)

=

-

E

x

~

μ

ref

,

T

~

μ

trans

[

ln

D

(

T

(

x

)

)

]

-

E

x

~

μ

G

[

ln

(

1

-

D

(

x

)

)

]

min

G

L

G

(

D

,

μ

G

)

=

-

E

x

~

μ

G

[

ln

(

1

-

D

(

x

)

)

]

$$\begin{cases} \min_D L_D(D, \mu_{\text{ref}}) = -\mathbb{E}_{x \sim \mu_{\text{ref}}} [\ln D(T(x))] \\ \min_G L_G(D, \mu_G) = -\mathbb{E}_{x \sim \mu_G} [\ln(1-D(x))] \end{cases}$$

This is equivalent to a GAN game with a different distribution

$$\mu_{\text{ref}},$$

$$\mu_{\text{ref}}$$

, sampled by

T

(

x

)

$$T(x)$$

, with

x

~

μ

ref

,

T

~

μ

trans

$$\{ \displaystyle x \sim \mu_{\text{ref}}, T \sim \mu_{\text{trans}} \}$$

. For example, if

μ

ref

$$\{ \displaystyle \mu_{\text{ref}} \}$$

is the distribution of images in ImageNet, and

μ

trans

$$\mu_{\text{trans}}$$

samples identity-transform with probability 0.5, and horizontal-reflection with probability 0.5,
then

μ

ref

,

$$\mu_{\text{ref}}$$

is the distribution of images in ImageNet and horizontally-reflected ImageNet, combined.

The result of such training would be a generator that mimics

$$\mu$$

$$\text{ref}$$

$$,$$

$$\mu_{\text{ref}}$$

. For example, it would generate images that look like they are randomly cropped, if the data augmentation uses random cropping.

The solution is to apply data augmentation to both generated and real images:

{

min

D

L

D

(

D

,

μ

G

)

=

-

E

x

~

μ

ref

,

T

~

μ

trans

[

ln

D

(

T

(

x

)

)

]

-

E

x

~

μ

G

,

T

~

μ

trans

[

ln

(

1

-

D

(

T

(

x

)

)

)

]

min

G

L

G

(

D

,

μ

G

)

=

-

E

x

~

μ

G

,

T

~

μ

trans

[

ln

(

1
-
D
(
T
(
x
)
)
)
]

$$\{\displaystyle \begin{cases} \min_{D} L_D(D, \mu_G) = -\operatorname{E}_{x \sim \mu_{\text{ref}}, T \sim \mu_{\text{trans}}} [\ln D(T(x))] - \operatorname{E}_{x \sim \mu_G, T \sim \mu_{\text{trans}}} [\ln(1 - D(T(x)))] \\ \min_{G} L_G(D, \mu_G) = -\operatorname{E}_{x \sim \mu_G, T \sim \mu_{\text{trans}}} [\ln(1 - D(T(x)))] \end{cases}$$

The authors demonstrated high-quality generation using just 100-picture-large datasets.

The StyleGAN-2-ADA paper points out a further point on data augmentation: it must be

invertible. Continue with the example of generating ImageNet pictures. If the data augmentation is "randomly rotate the picture by 0, 90, 180, 270 degrees with equal probability", then there is no way for the generator to know which is the true orientation: Consider two generators

G

,

G

,

$$\{G,G'\}$$

, such that for any latent

z

$$z$$

, the generated image

G

(

z

)

$G(z)$

is a 90-degree rotation of

G

,

(

z

)

$G'(z)$

. They would have exactly the same expected loss, and so neither is preferred over the other. The solution is to only use invertible data augmentation: instead of "randomly rotate the picture by 0, 90, 180, 270 degrees with equal probability", use "randomly rotate the picture by 90, 180, 270 degrees with 0.1 probability, and keep the picture as it is with 0.7 probability". This way, the generator is still rewarded to keep images oriented the same way as un-augmented ImageNet pictures.

Abstractly, the effect of randomly sampling transformations

T

:

Ω

→

Ω

$$\{\displaystyle T:\Omega \to \Omega \}$$

from the distribution

μ

trans

$$\mu_{\text{trans}}$$

is to define a Markov kernel

K

trans

:

Ω

$\rightarrow$

P

(
 Ω
)

$$K_{\text{trans}}(\Omega \rightarrow \mathcal{P}(\Omega))$$

. Then, the data-augmented GAN game pushes the generator to find some

μ
 $\wedge$

G

∈

P

(

Ω

)

$$\{\hat{\mu}\}_{G \in \mathcal{P}(\Omega)}$$

, such that

K

trans

*

μ

ref

=

K

trans

*

μ

^

G

$$K_{\text{trans}} \mu_{\text{ref}} = K_{\text{trans}} \hat{\mu}_G$$

where

$*$

$\{ \displaystyle *\}$

is the Markov kernel convolution.

A data-augmentation method is defined to be invertible if its Markov kernel

K

trans

$\{ \displaystyle K_{\{ \text{trans} \}} \}$

satisfies

K

trans

*

μ

=

K

trans

*

μ

,

$\Rightarrow$

μ

$=$

μ

,

$\forall$

μ

,

μ

,

$\in$

P

(

Ω

)

$$\{ \textstyle K_{\text{trans}}^* \mu = K_{\text{trans}}^* \mu \implies \mu = \mu \quad \forall \text{all}$$

$$\mu,\mu'\in\{\mathcal{P}(\Omega)\}$$

Immediately by definition, we see that composing multiple invertible data-augmentation methods results in yet another invertible method. Also by definition, if the data-augmentation method is invertible, then using it in a GAN game does not change the optimal strategy

$$\mu$$

$$\wedge$$

$$G$$

$$\hat{\mu}_G$$

for the generator, which is still

μ

ref

$$\mu_{\text{ref}}$$

.

There are two prototypical examples of invertible Markov kernels:

Discrete case: Invertible stochastic matrices, when

Ω

$$\Omega$$

is finite.

For example, if

Ω

=

{

↑

,

↓

,

←

,

→

}

$$\Omega = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$$

is the set of four images of an arrow, pointing in 4 directions, and the data augmentation is "randomly rotate the picture by 90, 180, 270 degrees with probability

p

$$p$$

, and keep the picture as it is with probability

$$(1-3p)$$

$$(1-3p)$$

", then the Markov kernel

$$K$$

$$\text{trans}$$

$$K_{\text{trans}}$$

can be represented as a stochastic matrix:

[

K

trans

]

=

[

(

1

-

3

p

)

p

p

p

p

(

1

-

3

p

)

p

p

p

p

(

1

-

3

p

)

p

p

p

p

(

1

-

3

p

)

]

$$[K_{\text{trans}}]=\begin{bmatrix}(1-3p)&p&p&p\\p&(1-3p)&p&p\\p&p&(1-3p)&p\\p&p&p&(1-3p)\end{bmatrix}$$

and

K

trans

$${\displaystyle K_{\text{trans}}}$$

is an invertible kernel iff

[

K

trans

]

$$[K_{\text{trans}}]$$

is an invertible matrix, that is,

p

$\neq$

1

$/$

4

$$p \neq 1/4$$

.

Continuous case: The gaussian kernel, when

$$\Omega$$

$$=$$

$$\mathbb{R}$$

$$n$$

$$\{\displaystyle \Omega =\mathbb{R} ^{n}\}$$

for some

$$n$$

$$\geq$$

$$1$$

$$\{\displaystyle n\geq 1\}$$

.

For example, if

Ω

=

$\mathbb{R}$

256

2

$$\Omega = \mathbb{R}^{256^2}$$

is the space of 256x256 images, and the data-augmentation method is "generate a gaussian noise

z

~

N

(

0

,

I

256

2

)

$$z \sim \mathcal{N}(0, I_{256^2})$$

, then add

$$\epsilon$$

$$z$$

$$\epsilon z$$

to the image", then

$$K$$

$$\text{trans}$$

$$K_{\text{trans}}$$

is just convolution by the density function of

N

$($

0

$,$

ϵ

2

I

256

2

)

$$\{\mathrm{N}\}(0,\epsilon^2I_{256^2})$$

. This is invertible, because convolution by a gaussian is just convolution by the heat kernel, so given any

μ

$\in$

P

(

R

n

)

$$\{\displaystyle \mu \in \{\mathrmcal {P}\}(\mathrmbb {R} ^{n})\}$$

, the convolved distribution

K

trans

*

μ

$$\{\displaystyle K_{\text{trans}}*\mu \}$$

can be obtained by heating up

R

n

$\mathbb{R}^n$

precisely according to

μ

μ

, then wait for time

ϵ

2

$/$

4

$$\epsilon^2/4$$

. With that, we can recover

μ

$$\mu$$

by running the heat equation backwards in time for

ϵ

2

/

4

$$\epsilon^2/4$$

.

More examples of invertible data augmentations are found in the paper.

SinGAN

SinGAN pushes data augmentation to the limit, by using only a single image as training data and performing data augmentation on it. The GAN architecture is adapted to this training method by using a multi-scale pipeline.

The generator

G

$\{\displaystyle G\}$

is decomposed into a pyramid of generators

G

=

G

1

◦

G

2

◦

...

◦

G

N

$$G=G_{\{1\}}\circ G_{\{2\}}\circ \cdots \circ G_{\{N\}}$$

, with the lowest one generating the image

G

N

(

Z

N

)

$$G_N(z_N)$$

at the lowest resolution, then the generated image is scaled up to

r

(

G

N

(

z

N

)

)

$$r(G_{\{N\}}(z_{\{N\}}))$$

, and fed to the next level to generate an image

G

N

-

1

(

z

N

-

1

+

r

(

G

N

(

z

N

)

)

)

$$\{\displaystyle G_{N-1}\{z_{N-1}+r(G_N\{z_N\})\}\}$$

at a higher resolution, and so on. The discriminator is decomposed into a pyramid as well.

StyleGAN series

The StyleGAN family is a series of architectures published by Nvidia's research division.

Progressive GAN

Progressive GAN is a method for training GAN for large-scale image generation stably, by growing a GAN generator from small to large scale in a pyramidal fashion. Like SinGAN, it decomposes the generator as

G

$=$

G

1

$\circ$

G

2

◦

...

◦

G

N

$$G=G_1\circ G_2\circ \cdots \circ G_N$$

, and the discriminator as

D

=

D

1

◦

D

2

◦

...

◦

D

N

$$D=D_{\{1\}}\circ D_{\{2\}}\circ \cdots \circ D_{\{N\}}$$

.

During training, at first only

G

N

,

D

N

$$G_{\{N\}}, D_{\{N\}}$$

are used in a GAN game to generate 4x4 images. Then

G

N

-

1

,

D

N

-

1

$$\{G_{N-1}, D_{N-1}\}$$

are added to reach the second stage of GAN game, to generate 8x8 images, and so on, until we reach a GAN game to generate 1024x1024 images.

To avoid shock between stages of the GAN game, each new layer is "blended in" (Figure 2 of the paper). For example, this is how the second stage GAN game starts:

Just before, the GAN game consists of the pair

G

N

,

D

N

$$\{G_N, D_N\}$$

generating and discriminating 4x4 images.

Just after, the GAN game consists of the pair

(

(

1

-

α

)

+

α

.

G

N

-

1

)

◦

u

◦

G

N

,

D

N

∘

d

∘

(

(

1

-

α

)

+

α

.

D

N

-

1

)

$$\{(1-\alpha)+\alpha \cdot G_{\{N-1\}}\}\circ u\circ G_{\{N\},D_{\{N\}}}\circ d\circ ((1-\alpha$$

$$)+\alpha \cdot D_{\{N-1\}}\}$$

generating and discriminating 8x8 images. Here, the functions

$$u$$

$$,$$

$$d$$

$${\displaystyle u,d}$$

are image up- and down-sampling functions, and

$$\alpha$$

$${\displaystyle \alpha }$$

is a blend-in factor (much like an alpha in image composing) that smoothly glides from 0 to 1.

StyleGAN-1

StyleGAN-1 is designed as a combination of Progressive GAN with neural style transfer.

The key architectural choice of StyleGAN-1 is a progressive growth mechanism, similar to Progressive GAN. Each generated image starts as a constant

4

×

4

×

512

$${\displaystyle 4\times 4\times 512}$$

array, and repeatedly passed through style blocks. Each style block applies a "style latent vector" via affine transform ("adaptive instance normalization"), similar to how neural style transfer uses Gramian matrix. It then adds noise, and normalize (subtract the mean, then divide by the variance).

At training time, usually only one style latent vector is used per image generated, but sometimes two ("mixing regularization") in order to encourage each style block to independently perform its stylization without expecting help from other style blocks (since they might receive an entirely different style latent vector).

After training, multiple style latent vectors can be fed into each style block. Those fed to the lower layers control the large-scale styles, and those fed to the higher layers control the fine-detail styles.

Style-mixing between two images

x

,

x

,

$\{x, x'\}$

can be performed as well. First, run a gradient descent to find

z

,

z

,

$\{z, z'\}$

such that

G

$($

z

$)$

$\approx$

x

$,$

G

$($

z

$,$

$)$

$\approx$

x

$,$

$$G(z) \approx x, G(z') \approx x'$$

. This is called "projecting an image back to style latent space". Then,

$$z$$

$$z$$

can be fed to the lower style blocks, and

$$z$$

$$,$$

$$z'$$

to the higher style blocks, to generate a composite image that has the large-scale style of

x

$$x$$

, and the fine-detail style of

x

,

$$x'$$

. Multiple images can also be composed this way.

StyleGAN-2

StyleGAN-2 improves upon StyleGAN-1, by using the style latent vector to transform the convolution layer's weights instead, thus solving the "blob" problem.

This was updated by the StyleGAN-2-ADA ("ADA" stands for "adaptive"), which uses invertible data augmentation as described above. It also tunes the amount of data augmentation applied by starting at zero, and gradually increasing it until an "overfitting heuristic" reaches a

target level, thus the name "adaptive".

StyleGAN-3

StyleGAN-3 improves upon StyleGAN-2 by solving the "texture sticking" problem, which can be seen in the official videos. They analyzed the problem by the Nyquist-Shannon sampling theorem, and argued that the layers in the generator learned to exploit the high-frequency signal in the pixels they operate upon.

To solve this, they proposed imposing strict lowpass filters between each generator's layers, so that the generator is forced to operate on the pixels in a way faithful to the continuous signals they represent, rather than operate on them as merely discrete signals. They further imposed rotational and translational invariance by using more signal filters. The resulting StyleGAN-3 is able to solve the texture sticking problem, as well as generating images that rotate and translate smoothly.

Other uses

Other than for generative and discriminative modelling of data, GANs have been used for other things.

GANs have been used for transfer learning to enforce the alignment of the latent feature space, such as in deep reinforcement learning. This works by feeding the embeddings of the source and target task to the discriminator which tries to guess the context. The resulting loss is then (inversely) backpropagated through the encoder.

Applications

Science

Iteratively reconstruct astronomical images

Simulate gravitational lensing for dark matter research.

Model the distribution of dark matter in a particular direction in space and to predict the gravitational lensing that will occur.

Model high energy jet formation and showers through calorimeters of high-energy physics experiments.

Approximate bottlenecks in computationally expensive simulations of particle physics experiments. Applications in the context of present and proposed CERN experiments have demonstrated the potential of these methods for accelerating simulation and/or improving simulation fidelity.

Reconstruct velocity and scalar fields in turbulent flows.

GAN-generated molecules were validated experimentally in mice.

Medical

One of the major concerns in medical imaging is preserving patient privacy. Due to these reasons, researchers often face difficulties in obtaining medical images for their research purposes. GAN has been used for generating synthetic medical images, such as MRI and PET images to address this challenge.

GAN can be used to detect glaucomatous images helping the early diagnosis which is essential to avoid partial or total loss of vision.

GANs have been used to create forensic facial reconstructions of deceased historical figures.

Malicious

Concerns have been raised about the potential use of GAN-based human image synthesis for sinister purposes, e.g., to produce fake, possibly incriminating, photographs and videos.

GANs can be used to generate unique, realistic profile photos of people who do not exist, in order to automate creation of fake social media profiles.

In 2019 the state of California considered and passed on October 3, 2019, the bill AB-602,

which bans the use of human image synthesis technologies to make fake pornography without the consent of the people depicted, and bill AB-730, which prohibits distribution of manipulated videos of a political candidate within 60 days of an election. Both bills were authored by Assembly member Marc Berman and signed by Governor Gavin Newsom. The laws went into effect in 2020.

DARPA's Media Forensics program studies ways to counteract fake media, including fake media produced using GANs.

Fashion, art and advertising

GANs can be used to generate art; The Verge wrote in March 2019 that "The images created by GANs have become the defining look of contemporary AI art." GANs can also be used to

inpaint photographs

generate fashion models, shadows, photorealistic renders of interior design, industrial design, shoes, etc. Such networks were reported to be used by Facebook.

Some have worked with using GAN for artistic creativity, as "creative adversarial network". A GAN, trained on a set of 15,000 portraits from WikiArt from the 14th to the 19th century, created the 2018 painting Edmond de Belamy, which sold for US\$432,500.

GANs were used by the video game modding community to up-scale low-resolution 2D textures in old video games by recreating them in 4k or higher resolutions via image training, and then down-sampling them to fit the game's native resolution (resembling supersampling anti-aliasing).

In 2020, Artbreeder was used to create the main antagonist in the sequel to the psychological web horror series Ben Drowned. The author would later go on to praise GAN applications for their ability to help generate assets for independent artists who are short on budget and manpower.

In May 2020, Nvidia researchers taught an AI system (termed "GameGAN") to recreate the game of Pac-Man simply by watching it being played.

In August 2019, a large dataset consisting of 12,197 MIDI songs each with paired lyrics and melody alignment was created for neural melody generation from lyrics using conditional GAN-LSTM (refer to sources at GitHub AI Melody Generation from Lyrics).

Miscellaneous

GANs have been used to

show how an individual's appearance might change with age.

reconstruct 3D models of objects from images,

generate novel objects as 3D point clouds,

model patterns of motion in video.

inpaint missing features in maps, transfer map styles in cartography or augment street view imagery.

use feedback to generate images and replace image search systems.

visualize the effect that climate change will have on specific houses.

reconstruct an image of a person's face after listening to their voice.

produces videos of a person speaking, given only a single photo of that person.

recurrent sequence generation.

History

In 1991, Juergen Schmidhuber published "artificial curiosity", neural networks in a zero-sum game. The first network is a generative model that models a probability distribution over output patterns. The second network learns by gradient descent to predict the reactions of the environment to these patterns. GANs can be regarded as a case where the environmental

reaction is 1 or 0 depending on whether the first network's output is in a given set.

Other people had similar ideas but did not develop them similarly. An idea involving adversarial networks was published in a 2010 blog post by Olli Niemitalo. This idea was never implemented and did not involve stochasticity in the generator and thus was not a generative model. An idea similar to GANs was used to model animal behavior by Wei Li, Melvin Gauci and Roderich Gross in 2013.

Another inspiration for GANs was noise-contrastive estimation, which uses the same loss function as GANs and which Goodfellow studied during his PhD in 2010–2014.

Adversarial machine learning has other uses besides generative modeling and can be applied to models other than neural networks. In control theory, adversarial learning based on neural networks was used in 2006 to train robust controllers in a game theoretic sense, by alternating the iterations between a minimizer policy, the controller, and a maximizer policy, the disturbance.

In 2017, a GAN was used for image enhancement focusing on realistic textures rather than pixel-accuracy, producing a higher image quality at high magnification. In 2017, the first faces were generated. These were exhibited in February 2018 at the Grand Palais. Faces generated by StyleGAN in 2019 drew comparisons with Deepfakes.

See also

Artificial intelligence art – Visual content generation through artificial intelligence
Pages displaying short descriptions of redirect targets

Deepfake – Realistic artificially generated media

Deep learning – Branch of machine learning

Diffusion model – Technique for the generative modeling of a continuous probability distribution

Generative AI – AI that generates content

Synthetic media – Artificial production of media by automated means

References

External links

Knight, Will. "5 Big Predictions for Artificial Intelligence in 2017". MIT Technology Review. Retrieved January 5, 2017.

Karras, Tero; Laine, Samuli; Aila, Timo (2018). "A Style-Based Generator Architecture for Generative Adversarial Networks". arXiv:1812.04948 [cs.NE].

This Person Does Not Exist – photorealistic images of people who do not exist, generated by StyleGAN

This Cat Does Not Exist Archived March 5, 2019, at the Wayback Machine – photorealistic images of cats who do not exist, generated by StyleGAN

Wang, Zhengwei; She, Qi; Ward, Tomas E. (2019). "Generative Adversarial Networks in Computer Vision: A Survey and Taxonomy". arXiv:1906.01529 [cs.LG].